Why People Hate AppSec

…and the Tools They Ride In On

Nate Larson
Freelance Security Consultant

About the Speaker

- 20ish years developing insecure software
- 15ish years in AppSec
- BA CSci, MS SE, CISSP
- Chickens, astronomy, cribbage

To devs, AppSec can come across less like a partner and more like a Vogon:

“Not actually evil, but bad-tempered, bureaucratic, officious and callous.”

D. Adams, The Hitchhiker’s Guide to the Galaxy. London: Pan Books, 1979.

The build Is broken

…and it’s not the scanner’s fault

Let's begin with a [slightly fictionalized] story…

Security wants fewer risks. Devs want fewer blockers.

Why are these treated as opposites?

Why devs push back

…or totally blow AppSec off

- Too many tools!
- Notification fatigue
- Backlog fatigue
- Meeting fatigue

How we try to help

...but sometimes utterly fail?

- Tool integration
- Trainings
- Metrics

The psychology of security friction

- Incentive misalignment
- The “gate” mental model
- Alert fatigue and tool noise
- The vocabulary gap

The BISS Model

“Because I Said So”

Or:

“Because the tool reports it”

“Because… Metrics”

What’s at stake

Framing risk in terms development leads care about

- Threat landscape shifts faster than SDLC was designed for
- Cost of late-cycle vs. early-cycle remediation *
- Regulatory and liability pressure is increasing
- Key reframe: it’s not AppSec’s problem alone

* T. Anderson, "Everyone cites that 'bugs are 100x more expensive to fix in production' research, but the study might not even exist," The Register, Jul. 22, 2021. [Online]. Available: https://www.theregister.com/2021/07/22/bugs_expense_bs. [Accessed: Apr. 23, 2026].

The pipeline is your friend

What “integrated” really means vs. what it usually looks like

Maturity levels: where does your org sit?

0. “We pass zipped source files around.”
1. “We use git, but no security tools are involved.”
2. “Scanning happens before Prod; we have a time-to-fix policy.”
3. “Scanning happens before Prod, and blocks the build.”
4. “All our tools are in the devs’ IDEs.”

Principles for Low-Friction Tooling

Some distinctly anti-Vogon ideals:

- Surface findings where developers work
- Signal-to-noise tuning
- Actionable output
- Show dev teams the tool is in their best interest
- Be reasonable

Enshiftifying the SDLC

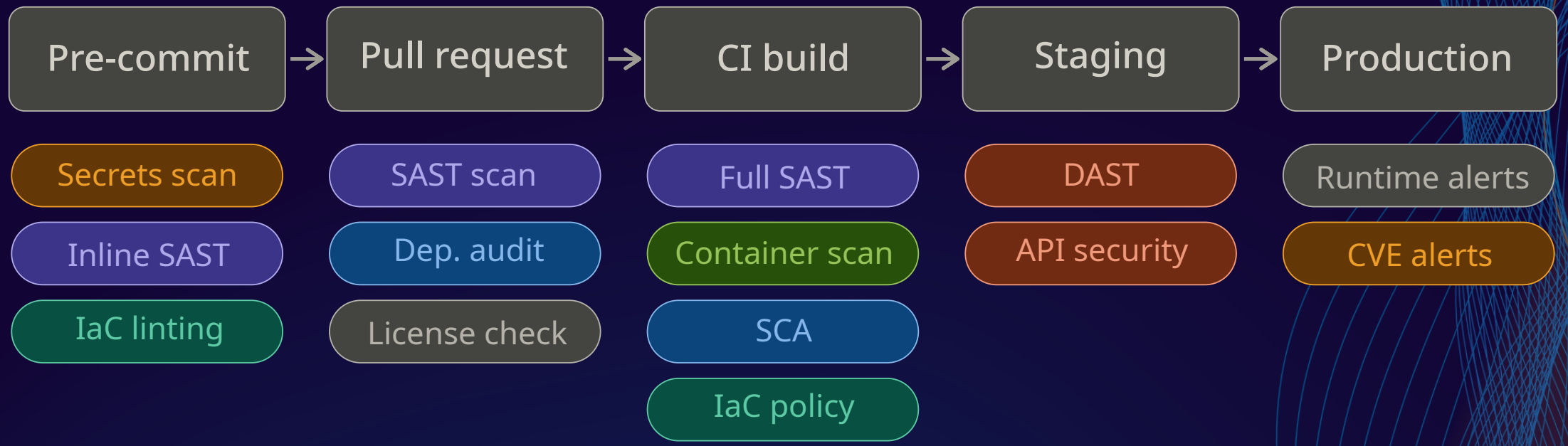
Shift left — but don’t throw everything into pre-commit.

Pre-commit checks should be: quick, low false-positives, actionable immediately.

Some good candidates:

- Secrets scanning
- Highly focused, lightweight SAST
- IaC linting

Security in the CI/CD pipeline



Building bridges: teams

How to speak with:

- **Developers:** Lead with empathy, not compliance.
- **Engineering leads:** Translate risk into delivery risk.
- **Leadership:** Connect scanning to business outcomes.

Building bridges: ideas

Things to try:

- Lightweight friction audit
- Security champion model
- Recognition-first metrics

A pragmatic framework

AppSec as enabler: a model you can take back next week

- **People:** Not "security says no"; "security helps us ship safer".
- **Process:** Define clear ownership.
- **Tools:** Integrate findings into developer workflows.

Adaptation as a core skill

The organizations that adapt fastest
aren’t the ones with the most tools

they’re the ones that removed the organizational friction
preventing good security hygiene.

Conversation starters for next week

With developers	What’s the one thing our tools do that wastes the most of your time?
With engineering leads	Let’s spend 20 minutes mapping which security issues are actual delivery risk.
With leadership	We’re spending a lot on tools. Let’s talk about whether we’re spending on the right friction points.

The goal isn’t a frictionless AppSec program.
It’s friction in the right places
and acceleration everywhere else.

In other words, an AppSec program that
doesn’t read like Vogon poetry.

THANK YOU

SECURE360

Keep in touch

xenloops@protonmail.com